

Pruebas Unitarias Backend

Cristian Esteban Agualimpia Torres
Luis Nicolás Calderón Correa
Gabriel Espitia Romero
Juanita Franco Sánchez
Andrés Felipe Ruge Passito

Pontificia Universidad Javeriana
Desarrollo Web
Bogotá D.C.
21 de octubre del 2024

Test Controladores

- **ControllerArrendadorTest**

La clase ControllerArrendadorTest contiene pruebas unitarias para los métodos del controlador ControllerArrendador. Estas pruebas verifican las operaciones básicas del CRUD (Crear, Leer, Actualizar, Eliminar) para el recurso Arrendador, asegurando que el controlador maneje correctamente las solicitudes HTTP y responda según lo esperado.

- **testGetAllArrendadores:**

- **Propósito:** Verificar que el método GET /arrendador/arrendadores devuelva la lista completa de arrendadores en formato JSON.
- **Proceso:** Se simulan dos objetos DTOArrendador, se configura el servicio para que los retorne, y se realiza una solicitud GET verificando que la respuesta sea correcta.
- **Resultado Esperado:** Código de estado 200 (OK) y una lista de arrendadores en formato JSON.

- **testGetArrendador**

- **Propósito:** Verificar que el método GET /arrendador/arrendador/{id} devuelva el arrendador correspondiente al ID proporcionado.
- **Proceso:** Se configura el servicio para devolver un DTOArrendador específico basado en el ID, y se verifica la respuesta.
- **Resultado Esperado:** Código de estado 200 (OK) y el arrendador solicitado en formato JSON.

- **testCrearArrendador**

- **Propósito:** Verificar que el método POST /arrendador/crearArrendador cree un nuevo arrendador.
- **Proceso:** Se configura el servicio para aceptar y devolver un DTOArrendador creado. Se envía un JSON representando el arrendador y se verifica la respuesta.
- **Resultado Esperado:** Código de estado 200 (OK) y el objeto arrendador recién creado en formato JSON.

- **testActualizarArrendador**

- **Propósito:** Verificar que el método PUT /arrendador/actualizarArrendador/{id} actualice los datos de un arrendador existente.
- **Proceso:** Se configura el servicio para aceptar y devolver un DTOArrendador actualizado. Se envía un JSON con la información actualizada y se verifica la respuesta.

- **Resultado Esperado:** Código de estado 200 (OK) y el arrendador actualizado en formato JSON.

- **testEliminarArrendador**

- **Propósito:** Verificar que el método DELETE `/arrendador/eliminarArrendador/{id}` elimine un arrendador específico.
- **Proceso:** Se simula la eliminación de un arrendador y se verifica la respuesta del controlador.
- **Resultado Esperado:** Código de estado 200 (OK) y un mensaje de confirmación ("Arrendador eliminado con éxito.").

- **testEliminarArrendadorNotFound**

- **Propósito:** Verificar que se maneje correctamente el caso en que se intente eliminar un arrendador inexistente.
- **Proceso:** Se configura el servicio para lanzar una excepción `EntityNotFoundException`, y se verifica que la respuesta sea un código 404 (Not Found).
- **Resultado Esperado:** Código de estado 404 (Not Found) y un mensaje de error ("Arrendador no encontrado.").

- **testEliminarArrendadorError**

- **Propósito:** Verificar que se manejen errores internos del servidor durante la eliminación de un arrendador.
- **Proceso:** Se configura el servicio para lanzar una `RuntimeException`, y se verifica que la respuesta sea un código 500 (Internal Server Error).
- **Resultado Esperado:** Código de estado 500 (Internal Server Error) y un mensaje de error ("Error al eliminar el arrendador.").

- **ControllerArrendatarioTest**

La clase `ControllerArrendatarioTest` contiene pruebas unitarias para los métodos del controlador `ControllerArrendatario`. Estas pruebas verifican las operaciones básicas del CRUD (Crear, Leer, Actualizar, Eliminar) para el recurso `Arrendatario`, asegurando que el controlador maneje correctamente las solicitudes HTTP y responda según lo esperado.

- **testGetAllArrendatarios**

- **Propósito:** Verificar que el método GET `/arrendatario/arrendatarios` devuelva la lista completa de arrendatarios en formato JSON.
- **Proceso:** Simular dos objetos `DTOArrendatario`, configurar el servicio para devolverlos, enviar solicitud GET y verificar que la respuesta sea correcta.

- **Resultado Esperado:** Código 200 (OK) y lista de arrendatarios en formato JSON.
- **testGetArrendatario**
 - **Propósito:** Verificar que el método GET /arrendatario/arrendatario/{id} devuelva el arrendatario correspondiente al ID proporcionado.
 - **Proceso:** Configurar el servicio para devolver un DTOArrendatario específico basado en el ID, enviar solicitud GET y verificar la respuesta.
 - **Resultado Esperado:** Código 200 (OK) y el arrendatario solicitado en formato JSON.
- **testCrearArrendatario**
 - **Propósito:** Verificar que el método POST /arrendatario/crearArrendatario cree un nuevo arrendatario.
 - **Proceso:** Configurar el servicio para aceptar y devolver un DTOArrendatario creado, enviar un JSON con los datos del arrendatario y verificar la respuesta.
 - **Resultado Esperado:** Código 200 (OK) y el objeto arrendatario recién creado en formato JSON.
- **testActualizarArrendatario**
 - **Propósito:** Verificar que el método PUT /arrendatario/actualizarArrendatario/{id} actualice los datos de un arrendatario existente.
 - **Proceso:** Configurar el servicio para aceptar y devolver un DTOArrendatario actualizado, enviar un JSON con la información actualizada y verificar la respuesta.
 - **Resultado Esperado:** Código 200 (OK) y el arrendatario actualizado en formato JSON.
- **testEliminarArrendatario**
 - **Propósito:** Verificar que el método DELETE /arrendatario/eliminarArrendatario/{id} elimine un arrendatario específico.
 - **Proceso:** Simular la eliminación de un arrendatario y verificar la respuesta del controlador.
 - **Resultado Esperado:** Código 200 (OK) y mensaje de confirmación ("Arrendatario eliminado con éxito.").
- **testEliminarArrendatarioNotFound**

- **Propósito:** Verificar que se maneje correctamente el caso en que se intente eliminar un arrendatario inexistente.
- **Proceso:** Configurar el servicio para lanzar una excepción `EntityNotFoundException`, enviar solicitud `DELETE` y verificar que la respuesta sea un código 404 (Not Found).
- **Resultado Esperado:** Código 404 (Not Found) y mensaje de error ("Arrendatario no encontrado").

- **testEliminarArrendatarioError**

- **Propósito:** Verificar que se manejen errores internos del servidor durante la eliminación de un arrendatario.
- **Proceso:** Configurar el servicio para lanzar una `RuntimeException`, enviar solicitud `DELETE` y verificar que la respuesta sea un código 500 (Internal Server Error).
- **Resultado Esperado:** Código 500 (Internal Server Error) y mensaje de error ("Error al eliminar el Arrendatario").

- **ControllerCalificacionTest**

La clase `ControllerCalificacionTest` contiene pruebas unitarias para los métodos del controlador `ControllerCalificacion`. Estas pruebas verifican las operaciones básicas del CRUD (Crear, Leer, Actualizar, Eliminar) para el recurso Calificación, asegurando que el controlador maneje correctamente las solicitudes HTTP y responda de acuerdo con lo esperado.

- **testGetAllCalificaciones**

- **Propósito:** Verificar que el método `GET /calificacion/calificaciones` devuelva todas las calificaciones.
- **Proceso:** Se crean dos objetos `DTOCalificacion`, se simula el servicio para devolverlos, y se envía la solicitud `GET`.
- **Resultado Esperado:** Código 200 (OK) y la lista de calificaciones en formato JSON.

- **testGetCalificacionById**

- **Propósito:** Verificar que el método `GET /calificacion/calificaciones/{id}` devuelva la calificación correspondiente al ID proporcionado.
- **Proceso:** Se simula el servicio para devolver un `DTOCalificacion` específico basado en el ID.
- **Resultado Esperado:** Código 200 (OK) y la calificación solicitada en formato JSON.

- **testgetCalificacionesArrendador**

- **Propósito:** Verificar que el método GET `/calificacion/arrendador/{id}/calificaciones` devuelva calificaciones de un arrendador específico.
- **Proceso:** Se crean y simulan calificaciones de un arrendador y se envía la solicitud GET.
- **Resultado Esperado:** Código 200 (OK) y la lista de calificaciones del arrendador en formato JSON.

- **testgetCalificacionesArrendatario**

- **Propósito:** Verificar que el método GET `/calificacion/arrendatario/{id}/calificaciones` devuelva calificaciones de un arrendatario específico.
- **Proceso:** Similar al anterior, pero para arrendatarios.
- **Resultado Esperado:** Código 200 (OK) y la lista de calificaciones del arrendatario en formato JSON.

- **testgetCalificacionesPropiedad**

- **Propósito:** Verificar que el método GET `/calificacion/propiedad/{id}/calificaciones` devuelva calificaciones de una propiedad específica.
- **Proceso:** Se crean y simulan calificaciones de una propiedad.
- **Resultado Esperado:** Código 200 (OK) y la lista de calificaciones de la propiedad en formato JSON.

- **testcrearCalificacion**

- **Propósito:** Verificar que el método POST `/calificacion/crearCalificacion` cree una nueva calificación.
- **Proceso:** Se envía un JSON con los datos de la calificación y se verifica la respuesta.
- **Resultado Esperado:** Código 200 (OK) y el objeto de calificación recién creado en formato JSON.

- **testActualizarCalificacion**

- **Propósito:** Verificar que el método PUT `/calificacion/actualizarCalificacion/{id}` actualice una calificación existente.
- **Proceso:** Se envía un JSON con la información actualizada y se verifica la respuesta.

- **Resultado Esperado:** Código 200 (OK) y la calificación actualizada en formato JSON.

- **testEliminarCalificacion**

- **Propósito:** Verificar que el método DELETE /calificacion/eliminarCalificaciones/{id} elimine una calificación específica.
- **Proceso:** Se envía la solicitud DELETE y se verifica la respuesta.
- **Resultado Esperado:** Código 200 (OK) y mensaje de confirmación.

- **testEliminarCalificacionNotFound**

- **Propósito:** Verificar que se maneje correctamente la eliminación de una calificación inexistente.
- **Proceso:** Se lanza una excepción EntityNotFoundException al intentar eliminar una calificación.
- **Resultado Esperado:** Código 404 (Not Found) y mensaje de error.

- **testEliminarCalificacionError**

- **Propósito:** Verificar que se manejen errores internos del servidor durante la eliminación de una calificación.
- **Proceso:** Se lanza una RuntimeException al intentar eliminar la calificación.
- **Resultado Esperado:** Código 500 (Internal Server Error) y mensaje de error.

- **ControllerPropiedadTest**

La clase ControllerPropiedadTest contiene pruebas unitarias para los métodos del controlador ControllerPropiedad. Estas pruebas verifican las operaciones básicas del CRUD (Crear, Leer, Actualizar, Eliminar) para el recurso Propiedad, asegurando que el controlador maneje correctamente las solicitudes HTTP y responda de acuerdo con lo esperado.

- **testTraerPropiedades**

- **Propósito:** Verificar que el método GET /propiedad/propiedades devuelva la lista completa de propiedades en formato JSON.
- **Proceso:** Se crean dos objetos DTOPropiedad simulando las propiedades, se configura el servicio para devolver la lista de propiedades, se envía la solicitud GET y se verifica la respuesta.
- **Resultado Esperado:** Código 200 (OK) y lista de propiedades en formato JSON.

- **testObtenerPropiedad**

- **Propósito:** Verificar que el método GET /propiedad/propiedad/{id} devuelva la propiedad correspondiente al ID proporcionado.
- **Proceso:** Se simula el servicio para devolver un DTOPropiedad basado en el ID proporcionado, se envía la solicitud GET y se verifica la respuesta.
- **Resultado Esperado:** Código 200 (OK) y la propiedad solicitada en formato JSON.
- **testObtenerPropiedadesPorCamas**
 - **Propósito:** Verificar que el método GET /propiedad/camas/{numero} devuelva una lista de propiedades que tengan el número de camas especificado.
 - **Proceso:** Se crea una lista de DTOPropiedad con una propiedad que tiene el número de camas especificado, se simula el servicio para devolver esta lista y se envía la solicitud GET.
 - **Resultado Esperado:** Código 200 (OK) y la lista de propiedades que cumplen con el criterio en formato JSON.
- **testObtenerPropiedadesDisponibles**
 - **Propósito:** Verificar que el método GET /propiedad/disponibles devuelva la lista de propiedades disponibles.
 - **Proceso:** Se simula una lista de propiedades disponibles usando DTOPropiedad, se configura el servicio para devolver la lista y se envía la solicitud GET.
 - **Resultado Esperado:** Código 200 (OK) y la lista de propiedades disponibles en formato JSON.
- **testObtenerPropiedadesPorCuartos**
 - **Propósito:** Verificar que el método GET /propiedad/cuartos/{numero} devuelve una lista de propiedades que tengan el número de cuartos especificado.
 - **Proceso:** Se simula una lista de DTOPropiedad con propiedades que tienen el número de cuartos especificado, se configura el servicio para devolver esta lista y se envía la solicitud GET.
 - **Resultado Esperado:** Código 200 (OK) y la lista de propiedades con el número de cuartos indicado en formato JSON.
- **testObtenerPropiedadesPorArea**
 - **Propósito:** Verificar que el método GET /propiedad/area/{area} devuelva las propiedades que tienen el área especificada.
 - **Proceso:** Se crea un objeto DTOPropiedad con los datos necesarios y se simula una lista de propiedades que coincide con el área solicitada. Luego, se envía una solicitud GET a la URL correspondiente con el área deseada.

- **Resultado Esperado:** Se espera un código 200 (OK) y una lista de propiedades en formato JSON que contenga la propiedad con el área especificada.
- **testObtenerPropiedadesPorCapacidad**
 - **Propósito:** Verificar que el método GET /propiedad/capacidad/{capacidad} devuelva las propiedades que tienen la capacidad especificada.
 - **Proceso:** Se crea un objeto DTOPropiedad con los datos relevantes y se simula una lista de propiedades que cumplen con la capacidad solicitada. A continuación, se envía una solicitud GET a la URL correspondiente con la capacidad deseada.
 - **Resultado Esperado:** Se espera un código 200 (OK) y una lista de propiedades en formato JSON que incluya la propiedad con la capacidad especificada.
- **testGetPropiedadesPorArrendadorId**
 - **Propósito:** Verificar que el método GET /propiedad/arrendador/{id}/propiedades devuelva todas las propiedades asociadas al arrendador con el ID proporcionado.
 - **Proceso:** Se crea un objeto DTOPropiedad que representa una propiedad perteneciente a un arrendador específico y se simula la lista de propiedades para ese arrendador. Luego, se envía una solicitud GET a la URL correspondiente con el ID del arrendador.
 - **Resultado Esperado:** Se espera un código 200 (OK) y una lista de propiedades en formato JSON que contenga la propiedad asociada al arrendador especificado.
- **testCrearPropiedad**
 - **Propósito:** Verificar que el método POST /propiedad/crearPropiedad cree una nueva propiedad.
 - **Proceso:** Se crea un objeto DTOPropiedad con los datos relevantes y se simula el servicio para que retorne este objeto cuando se intente crear. Se envía una solicitud POST con un JSON que contiene los datos necesarios para crear la propiedad.
 - **Resultado Esperado:** Se espera un código 200 (OK) y el objeto de propiedad recién creado en formato JSON.
- **testActualizarPropiedad**
 - **Propósito:** Verificar que el método PUT /propiedad/actualizarPropiedad/{id} actualice correctamente una propiedad existente.
 - **Proceso:** Se crea un objeto DTOPropiedad con los datos actualizados y se simula el servicio para que retorne este objeto. Luego, se envía una solicitud PUT con los datos actualizados a la URL correspondiente.

- **Resultado Esperado:** Se espera un código 200 (OK) y el objeto de propiedad actualizado en formato JSON.

- **testEliminarPropiedad**

- **Propósito:** Verificar que el método DELETE /propiedad/eliminarPropiedad/{id} elimine correctamente una propiedad.
- **Proceso:** Se simula el servicio para que el método de eliminación retorne un mensaje de éxito. Luego, se envía una solicitud DELETE a la URL correspondiente con el ID de la propiedad a eliminar.
- **Resultado Esperado:** Se espera un código 200 (OK) y un mensaje que indique que la propiedad ha sido eliminada con éxito.

- **testEliminarPropiedadNotFound**

- **Propósito:** Verificar que el método DELETE /propiedad/eliminarPropiedad/{id} maneje correctamente el caso en que se intenta eliminar una propiedad que no existe.
- **Proceso:** Se simula el servicio para que lance una excepción EntityNotFoundException cuando se intente eliminar la propiedad con el ID especificado. Luego, se envía una solicitud DELETE a la URL correspondiente.
- **Resultado Esperado:** Se espera un código 404 (Not Found) y un mensaje que indique que la propiedad no fue encontrada.

- **testEliminarPropiedadError**

- **Propósito:** Verificar que el método DELETE /propiedad/eliminarPropiedad/{id} maneje correctamente errores internos del servidor.
- **Proceso:** Se simula el servicio para que lance una excepción RuntimeException al intentar eliminar la propiedad con el ID especificado. Luego, se envía una solicitud DELETE a la URL correspondiente.
- **Resultado Esperado:** Se espera un código 500 (Internal Server Error) y un mensaje que indique que hubo un error al eliminar la propiedad.

- **ControllerSolicitudTest**

La clase ControllerSolicitudTest contiene pruebas unitarias para los métodos del controlador ControllerSolicitud. Estas pruebas verifican las operaciones básicas del CRUD (Crear, Leer, Actualizar, Eliminar) para el recurso Solicitud, asegurando que el controlador maneje correctamente las solicitudes HTTP y responda de acuerdo con lo esperado.

- **testGetAllSolicitudes**

- **Propósito:** Verificar que el método GET /solicitud/solicitudes devuelva la lista completa de solicitudes en formato JSON.
- **Proceso:** Se crea un objeto DTOSolicitud simulando una solicitud, se configura el servicio para devolver una lista de solicitudes, y se envía la solicitud GET.
- **Resultado Esperado:** Código 200 (OK) y lista de solicitudes en formato JSON.
- **testGetSolicitud**
 - **Propósito:** Verificar que el método GET /solicitud/solicitud/{id} devuelva la solicitud correspondiente al ID proporcionado.
 - **Proceso:** Se simula el servicio para devolver un DTOSolicitud basado en el ID proporcionado, se envía la solicitud GET y se verifica la respuesta.
 - **Resultado Esperado:** Código 200 (OK) y la solicitud solicitada en formato JSON.
- **testGetSolicitudesByEstado**
 - **Propósito:** Verificar que el método GET /solicitud/solicitudes/estado/{estado} devuelva una lista de solicitudes que tengan el estado especificado.
 - **Proceso:** Se crea una lista de DTOSolicitud con una solicitud que tiene el estado especificado, se simula el servicio para devolver esta lista y se envía la solicitud GET.
 - **Resultado Esperado:** Código 200 (OK) y la lista de solicitudes que cumplen con el criterio en formato JSON.
- **testGetSolicitudesByArrendatarioId**
 - **Propósito:** Verificar que el método GET /solicitud/solicitudes/arrendatario/{id} devuelva todas las solicitudes asociadas al arrendatario con el ID proporcionado.
 - **Proceso:** Se simula una lista de DTOSolicitud que corresponde al arrendatario, se configura el servicio para devolver esta lista y se envía la solicitud GET.
 - **Resultado Esperado:** Código 200 (OK) y la lista de solicitudes en formato JSON asociadas al arrendatario especificado.
- **testGetSolicitudesByPropiedadId**
 - **Propósito:** Verificar que el método GET /solicitud/solicitudes/propiedad/{id} devuelva todas las solicitudes asociadas a la propiedad con el ID proporcionado.
 - **Proceso:** Se simula una lista de DTOSolicitud que corresponde a la propiedad, se configura el servicio para devolver esta lista y se envía la solicitud GET.
 - **Resultado Esperado:** Código 200 (OK) y la lista de solicitudes en formato JSON asociadas a la propiedad especificada.
- **testCrearSolicitud**

- **Propósito:** Verificar que el método POST /solicitud/crearSolicitud cree una nueva solicitud.
- **Proceso:** Se crea un objeto DTOSolicitud con los datos relevantes y se simula el servicio para que retorne este objeto cuando se intente crear. Se envía una solicitud POST con un JSON que contiene los datos necesarios para crear la solicitud.
- **Resultado Esperado:** Código 200 (OK) y el objeto de solicitud recién creado en formato JSON.
- **testActualizarSolicitud**
 - **Propósito:** Verificar que el método PUT /solicitud/actualizarSolicitud/{id} actualice correctamente una solicitud existente.
 - **Proceso:** Se crea un objeto DTOSolicitud con los datos actualizados y se simula el servicio para que retorne este objeto. Luego, se envía una solicitud PUT con los datos actualizados a la URL correspondiente.
 - **Resultado Esperado:** Código 200 (OK) y el objeto de solicitud actualizado en formato JSON.
- **testEliminarSolicitud**
 - **Propósito:** Verificar que el método DELETE /solicitud/eliminarSolicitud/{id} elimine correctamente una solicitud.
 - **Proceso:** Se simula el servicio para que el método de eliminación retorne un mensaje de éxito. Luego, se envía una solicitud DELETE a la URL correspondiente con el ID de la solicitud a eliminar.
 - **Resultado Esperado:** Código 200 (OK) y un mensaje que indique que la solicitud ha sido eliminada con éxito.
- **testEliminarSolicitudNotFound**
 - **Propósito:** Verificar que el método DELETE /solicitud/eliminarSolicitud/{id} maneje correctamente el caso en que se intenta eliminar una solicitud que no existe.
 - **Proceso:** Se simula el servicio para que lance una excepción EntityNotFoundException cuando se intente eliminar la solicitud con el ID especificado. Luego, se envía una solicitud DELETE a la URL correspondiente.
 - **Resultado Esperado:** Código 404 (Not Found) y un mensaje que indique que la solicitud no fue encontrada.
- **testEliminarSolicitudError**

- **Propósito:** Verificar que el método DELETE /solicitud/eliminarSolicitud/{id} maneje correctamente errores internos del servidor.
- **Proceso:** Se simula el servicio para que lance una excepción RuntimeException al intentar eliminar la solicitud con el ID especificado. Luego, se envía una solicitud DELETE a la URL correspondiente.
- **Resultado Esperado:** Código 500 (Internal Server Error) y un mensaje que indique que hubo un error al eliminar la solicitud.

- **ControllerUsuarioTest**

La clase ControllerUsuarioTest contiene pruebas unitarias para los métodos del controlador ControllerUsuario. Estas pruebas verifican las operaciones básicas del CRUD (Crear, Leer, Actualizar, Eliminar) para el recurso Usuario, asegurando que el controlador maneje correctamente las solicitudes HTTP y responda de acuerdo con lo esperado.

- **testGetAllUsuarios**

- **Propósito:** Verificar que el método GET /usuario/usuarios devuelva la lista completa de usuarios en formato JSON.
- **Proceso:** Se crean objetos DTOUsuario simulando usuarios, se configura el servicio para devolver una lista de usuarios, y se envía la solicitud GET.
- **Resultado Esperado:** Código 200 (OK) y lista de usuarios en formato JSON.

- **testGetUsuario**

- **Propósito:** Verificar que el método GET /usuario/usuario/{id} devuelva el usuario correspondiente al ID proporcionado.
- **Proceso:** Se simula el servicio para devolver un DTOUsuario basado en el ID proporcionado, se envía la solicitud GET y se verifica la respuesta.
- **Resultado Esperado:** Código 200 (OK) y el usuario solicitado en formato JSON.

- **testCrearUsuario**

- **Propósito:** Verificar que el método POST /usuario/crearUsuario cree un nuevo usuario.
- **Proceso:** Se crea un objeto DTOUsuario con los datos relevantes y se simula el servicio para que retorne este objeto cuando se intente crear. Se envía una solicitud POST con un JSON que contiene los datos necesarios para crear el usuario.
- **Resultado Esperado:** Código 200 (OK) y el objeto de usuario recién creado en formato JSON.

- **testActualizarUsuario**

- **Propósito:** Verificar que el método PUT /usuario/actualizarUsuario/{id} actualice correctamente un usuario existente.
 - **Proceso:** Se crea un objeto DTOUsuario con los datos actualizados y se simula el servicio para que retorne este objeto. Luego, se envía una solicitud PUT con los datos actualizados a la URL correspondiente.
 - **Resultado Esperado:** Código 200 (OK) y el objeto de usuario actualizado en formato JSON.
- **testEliminarUsuario**
 - **Propósito:** Verificar que el método DELETE /usuario/eliminarUsuario/{id} elimine correctamente un usuario.
 - **Proceso:** Se simula el servicio para que el método de eliminación retorne un mensaje de éxito. Luego, se envía una solicitud DELETE a la URL correspondiente con el ID del usuario a eliminar.
 - **Resultado Esperado:** Código 200 (OK) y un mensaje que indique que el usuario ha sido eliminado con éxito.
- **testEliminarUsuarioNotFound**
 - **Propósito:** Verificar que el método DELETE /usuario/eliminarUsuario/{id} maneje correctamente el caso en que se intenta eliminar un usuario que no existe.
 - **Proceso:** Se simula el servicio para que lance una excepción EntityNotFoundException cuando se intente eliminar el usuario con el ID especificado. Luego, se envía una solicitud DELETE a la URL correspondiente.
 - **Resultado Esperado:** Código 404 (Not Found) y un mensaje que indique que el usuario no fue encontrado.
- **testEliminarUsuarioError**
 - **Propósito:** Verificar que el método DELETE /usuario/eliminarUsuario/{id} maneje correctamente errores internos del servidor.
 - **Proceso:** Se simula el servicio para que lance una excepción RuntimeException al intentar eliminar el usuario con el ID especificado. Luego, se envía una solicitud DELETE a la URL correspondiente.
 - **Resultado Esperado:** Código 500 (Internal Server Error) y un mensaje que indique que hubo un error al eliminar el usuario.

Test Modelos

- **ArrendadorTest**

La clase ArrendadorTest contiene pruebas unitarias para los métodos de la clase Arrendador. Estas pruebas verifican la correcta creación y manipulación de objetos Arrendador, asegurando

que los getters y setters funcionen correctamente y que los objetos se inicialicen con los valores esperados.

- **testArrendadorCreation**

- **Propósito:** Verificar que un objeto Arrendador se cree correctamente con los valores iniciales esperados.
- **Proceso:** Se crea un objeto Arrendador en el método setUp con valores específicos para cada uno de sus atributos. Luego, se verifica que el objeto no sea nulo y que cada uno de sus atributos tenga el valor esperado.
- **Resultado Esperado:** El objeto Arrendador no es nulo y todos sus atributos tienen los valores esperados.

- **testSetAndGetPropiedades**

- **Propósito:** Verificar que el método setPropiedades y getPropiedades funcionen correctamente.
- **Proceso:** Se crea una lista de Propiedad y se asigna al objeto Arrendador usando el método setPropiedades. Luego, se verifica que el método getPropiedades devuelva la lista asignada.
- **Resultado Esperado:** El método getPropiedades devuelve la lista de propiedades asignada.

- **testSetAndGetCalificaciones**

- **Propósito:** Verificar que el método setCalificaciones y getCalificaciones funcionen correctamente.
- **Proceso:** Se crea una lista de Calificaciones y se asigna al objeto Arrendador usando el método setCalificaciones. Luego, se verifica que el método getCalificaciones devuelva la lista asignada.
- **Resultado Esperado:** El método getCalificaciones devuelve la lista de propiedades asignada.

- **ArrendatarioTest**

La clase ArrendatarioTest contiene pruebas unitarias para los métodos de la clase Arrendatario. Estas pruebas verifican la correcta creación y manipulación de objetos Arrendatario, asegurando que los getters y setters funcionen correctamente y que los objetos se inicialicen con los valores esperados.

- **testArrendatarioCreation**

- **Propósito:** Verificar que un objeto Arrendatario se cree correctamente con los valores iniciales esperados.

- **Proceso:** Se crea un objeto Arrendatario en el método setUp con valores específicos para cada uno de sus atributos. Luego, se verifica que el objeto no sea nulo y que cada uno de sus atributos tenga el valor esperado.
- **Resultado Esperado:** El objeto Arrendatario no es nulo y todos sus atributos tienen los valores esperados.

- **testSetAndGetSolicitudes**

- **Propósito:** Verificar que el método setSolicitudes y getSolicitudes funcionen correctamente.
- **Proceso:** Se crea una lista de Solicitud y se asigna al objeto Arrendatario usando el método setSolicitudes. Luego, se verifica que el método getSolicitudes devuelva la lista asignada.
- **Resultado Esperado:** El método getSolicitudes devuelve la lista de solicitudes asignada.

- **testSetAndGetCalificaciones**

- **Propósito:** Verificar que el método setCalificaciones y getCalificaciones funcionen correctamente.
- **Proceso:** Se crea una lista de Calificacion y se asigna al objeto Arrendatario usando el método setCalificaciones. Luego, se verifica que el método getCalificaciones devuelva la lista asignada.
- **Resultado Esperado:** El método getCalificaciones devuelve la lista de calificaciones asignada.

- **CalificacionTest**

La clase CalificacionTest contiene pruebas unitarias para los métodos de la clase Calificacion. Estas pruebas verifican la correcta creación y manipulación de objetos Calificacion, asegurando que los getters y setters funcionen correctamente y que los objetos se inicialicen con los valores esperados.

- **testCalificacionCreation**

- **Propósito:** Verificar que un objeto Calificacion se cree correctamente con los valores iniciales esperados.
- **Proceso:** Se crea un objeto Calificacion en el método setUp con valores específicos para cada uno de sus atributos. Luego, se verifica que el objeto no sea nulo y que cada uno de sus atributos tenga el valor esperado.
- **Resultado Esperado:** El objeto Calificacion no es nulo y todos sus atributos tienen los valores esperados.

- **testSetAndGetCalificacion**

- **Propósito:** Verificar que el método setCalificacion y getCalificacion funcionen correctamente.
- **Proceso:** Se establece un nuevo valor para el atributo calificacion usando el método setCalificacion. Luego, se verifica que el método getCalificacion devuelva el valor establecido.
- **Resultado Esperado:** El método getCalificacion devuelve el valor de calificación establecido.
- **testSetAndGetComentario**
 - **Propósito:** Verificar que el método setComentario y getComentario funcionen correctamente.
 - **Proceso:** Se establece un nuevo valor para el atributo comentario usando el método setComentario. Luego, se verifica que el método getComentario devuelva el valor establecido.
 - **Resultado Esperado:** El método getComentario devuelve el valor de comentario establecido.
- **testSetAndGetStatus**
 - **Propósito:** Verificar que el método setStatus y getStatus funcionen correctamente.
 - **Proceso:** Se establece un nuevo valor para el atributo status usando el método setStatus. Luego, se verifica que el método getStatus devuelva el valor establecido.
 - **Resultado Esperado:** El método getStatus devuelve el valor de status establecido.
- **testSetAndGetIdTipo**
 - **Propósito:** Verificar que el método setIdTipo y getIdTipo funcionen correctamente.
 - **Proceso:** Se establece un nuevo valor para el atributo idTipo usando el método setIdTipo. Luego, se verifica que el método getIdTipo devuelva el valor establecido.
 - **Resultado Esperado:** El método getIdTipo devuelve el valor de idTipo establecido.
- **testSetAndGetIdCalificado**
 - **Propósito:** Verificar que el método setIdCalificado y getIdCalificado funcionen correctamente.

- **Proceso:** Se establece un nuevo valor para el atributo idCalificado usando el método setIdCalificado. Luego, se verifica que el método getIdCalificado devuelva el valor establecido.
- **Resultado Esperado:** El método getIdCalificado devuelve el valor de idCalificado establecido.

- **PropiedadTest**

La clase PropiedadTest contiene pruebas unitarias para los métodos de la clase Propiedad. Estas pruebas verifican la correcta creación y manipulación de objetos Propiedad, asegurando que los getters y setters funcionen correctamente y que los objetos se inicialicen con los valores esperados.

- **testPropiedadCreation**

- **Propósito:** Verificar que un objeto Propiedad se cree correctamente con los valores iniciales esperados.
- **Proceso:** Se crea un objeto Propiedad en el método setUp con valores específicos para cada uno de sus atributos. Luego, se verifica que el objeto no sea nulo y que cada uno de sus atributos tenga el valor esperado.
- **Resultado Esperado:** El objeto Propiedad no es nulo y todos sus atributos tienen los valores esperados.

- **testSetAndGetNombre**

- **Propósito:** Verificar que el método setNombre y getNombre funcionen correctamente.
- **Proceso:** Se establece un nuevo valor para el atributo nombre usando el método setNombre. Luego, se verifica que el método getNombre devuelva el valor establecido.
- **Resultado Esperado:** El método getNombre devuelve el valor de nombre establecido.

- **testSetAndGetUbicacion**

- **Propósito:** Verificar que el método setUbicacion y getUbicacion funcionen correctamente.
- **Proceso:** Se establece un nuevo valor para el atributo ubicacion usando el método setUbicacion. Luego, se verifica que el método getUbicacion devuelva el valor establecido.
- **Resultado Esperado:** El método getUbicacion devuelve el valor de ubicación establecido.

- **testSetAndGetParqueadero**

- **Propósito:** Verificar que el método setParqueadero y isParqueadero funcionen correctamente.
- **Proceso:** Se establece un nuevo valor para el atributo parqueadero usando el método setParqueadero. Luego, se verifica que el método isParqueadero devuelva el valor establecido.
- **Resultado Esperado:** El método isParqueadero devuelve el valor de parqueadero establecido.
- **testSetAndGetCapacidad**
 - **Propósito:** Verificar que el método setCapacidad y getCapacidad funcionen correctamente.
 - **Proceso:** Se establece un nuevo valor para el atributo capacidad usando el método setCapacidad. Luego, se verifica que el método getCapacidad devuelva el valor establecido.
 - **Resultado Esperado:** El método getCapacidad devuelve el valor de capacidad establecido.
- **testSetAndGetDisponible**
 - **Propósito:** Verificar que el método setDisponible y isDisponible funcionen correctamente.
 - **Proceso:** Se establece un nuevo valor para el atributo disponible usando el método setDisponible. Luego, se verifica que el método isDisponible devuelva el valor establecido.
 - **Resultado Esperado:** El método isDisponible devuelve el valor de disponible establecido.
- **testSetAndGetPrecioXnoche**
 - **Propósito:** Verificar que el método setPrecioXnoche y getPrecioXnoche funcionen correctamente.
 - **Proceso:** Se establece un nuevo valor para el atributo precioXnoche usando el método setPrecioXnoche. Luego, se verifica que el método getPrecioXnoche devuelva el valor establecido.
 - **Resultado Esperado:** El método getPrecioXnoche devuelve el valor de precioXnoche establecido.
- **testSetAndGetStatus**
 - **Propósito:** Verificar que el método setStatus y getStatus funcionen correctamente.

- **Proceso:** Se establece un nuevo valor para el atributo status usando el método setStatus. Luego, se verifica que el método getStatus devuelva el valor establecido.
- **Resultado Esperado:** El método getStatus devuelve el valor de status establecido.

- **testSetAndGetArrendadorId**

- **Propósito:** Verificar que el método setArrendadorId y getArrendadorId funcionen correctamente.
- **Proceso:** Se establece un nuevo valor para el atributo arrendadorId usando el método setArrendadorId. Luego, se verifica que el método getArrendadorId devuelva el valor establecido.
- **Resultado Esperado:** El método getArrendadorId devuelve el valor de arrendadorId establecido.

- **testSetAndGetCalificaciones**

- **Propósito:** Verificar que el método setCalificaciones y getCalificaciones funcionen correctamente.
- **Proceso:** Se crea una lista de Calificacion y se asigna al objeto Propiedad usando el método setCalificaciones. Luego, se verifica que el método getCalificaciones devuelva la lista asignada.
- **Resultado Esperado:** El método getCalificaciones devuelve la lista de calificaciones asignada.

- **testSetAndGetSolicitudes**

- **Propósito:** Verificar que el método setSolicitudes y getSolicitudes funcionen correctamente.
- **Proceso:** Se crea una lista de Solicitud y se asigna al objeto Propiedad usando el método setSolicitudes. Luego, se verifica que el método getSolicitudes devuelva la lista asignada.
- **Resultado Esperado:** El método getSolicitudes devuelve la lista de solicitudes asignada.

- **SolicitudTest**

La clase SolicitudTest contiene pruebas unitarias para los métodos de la clase Solicitud. Estas pruebas verifican la correcta creación y manipulación de objetos Solicitud, asegurando que los getters y setters funcionen correctamente y que los objetos se inicialicen con los valores esperados.

- **testSolicitudCreation**

- **Propósito:** Verificar que un objeto Solicitud se cree correctamente con los valores iniciales esperados.
- **Proceso:** Se crea un objeto Solicitud en el método setUp con valores específicos para cada uno de sus atributos. Luego, se verifica que el objeto no sea nulo y que cada uno de sus atributos tenga el valor esperado.
- **Resultado Esperado:** El objeto Solicitud no es nulo y todos sus atributos tienen los valores esperados.
- **testSetAndGetFechaLlegada**
 - **Propósito:** Verificar que el método setFechaLlegada y getFechaLlegada funcionen correctamente.
 - **Proceso:** Se establece un nuevo valor para el atributo fechaLlegada usando el método setFechaLlegada. Luego, se verifica que el método getFechaLlegada devuelva el valor establecido.
 - **Resultado Esperado:** El método getFechaLlegada devuelve el valor de fechaLlegada establecido.
- **testSetAndGetFechaPartida**
 - **Propósito:** Verificar que el método setFechaPartida y getFechaPartida funcionen correctamente.
 - **Proceso:** Se establece un nuevo valor para el atributo fechaPartida usando el método setFechaPartida. Luego, se verifica que el método getFechaPartida devuelva el valor establecido.
 - **Resultado Esperado:** El método getFechaPartida devuelve el valor de fechaPartida establecido.
- **testSetAndGetAceptacion**
 - **Propósito:** Verificar que el método setAceptacion y isAceptacion funcionen correctamente.
 - **Proceso:** Se establece un nuevo valor para el atributo aceptacion usando el método setAceptacion. Luego, se verifica que el método isAceptacion devuelva el valor establecido.
 - **Resultado Esperado:** El método isAceptacion devuelve el valor de aceptacion establecido.
- **testSetAndGetCantidadPersonas**
 - **Propósito:** Verificar que el método setCantidadPersonas y getCantidadPersonas funcionen correctamente.

- **Proceso:** Se establece un nuevo valor para el atributo cantidadPersonas usando el método setCantidadPersonas. Luego, se verifica que el método getCantidadPersonas devuelva el valor establecido.
- **Resultado Esperado:** El método getCantidadPersonas devuelve el valor de cantidadPersonas establecido.

- **testSetAndGetStatus**

- **Propósito:** Verificar que el método setStatus y getStatus funcionen correctamente.
- **Proceso:** Se establece un nuevo valor para el atributo status usando el método setStatus. Luego, se verifica que el método getStatus devuelva el valor establecido.
- **Resultado Esperado:** El método getStatus devuelve el valor de status establecido.

- **testSetAndGetPropiedadId**

- **Propósito:** Verificar que el método setPropiedadId y getPropiedadId funcionen correctamente.
- **Proceso:** Se establece un nuevo valor para el atributo propiedadId usando el método setPropiedadId. Luego, se verifica que el método getPropiedadId devuelva el valor establecido.
- **Resultado Esperado:** El método getPropiedadId devuelve el valor de propiedadId establecido.

- **testSetAndGetArrendatarioId**

- **Propósito:** Verificar que el método setArrendatarioId y getArrendatarioId funcionen correctamente.
- **Proceso:** Se establece un nuevo valor para el atributo arrendatarioId usando el método setArrendatarioId. Luego, se verifica que el método getArrendatarioId devuelva el valor establecido.
- **Resultado Esperado:** El método getArrendatarioId devuelve el valor de arrendatarioId establecido.

- **UsuarioTest**

La clase UsuarioTest contiene pruebas unitarias para los métodos de la clase Usuario. Estas pruebas verifican la correcta creación y manipulación de objetos Usuario, asegurando que los getters y setters funcionen correctamente y que los objetos se inicialicen con los valores esperados.

- **setUp**

- **Propósito:** Configurar el objeto Usuario antes de cada prueba.
- **Proceso:** Se crea un objeto Usuario y se inicializan sus atributos con valores específicos.
- **Resultado Esperado:** El objeto Usuario se inicializa correctamente con los valores especificados.

- **testUsuarioCreation**

- **Propósito:** Verificar que un objeto Usuario se cree correctamente con los valores iniciales esperados.
- **Proceso:** Se verifica que el objeto Usuario no sea nulo y que cada uno de sus atributos tenga el valor esperado.
- **Resultado Esperado:** El objeto Usuario no es nulo y todos sus atributos tienen los valores esperados.

Test servicios

- **ServicioArrendadorTest**

La clase ServicioArrendadorTest contiene pruebas unitarias para los métodos de la clase ServicioArrendador. Estas pruebas verifican la correcta funcionalidad de los métodos de la clase, asegurando que se comporten como se espera en diferentes escenarios.

- **testTraerArrendadores**

- **Propósito:** Verificar que el método traerArrendadores devuelva correctamente una lista de arrendadores existentes.
- **Proceso:** Se crean varios objetos Arrendador y se configura el repositorio para devolver una lista que contiene estos objetos cuando se llama al método findAll. Luego, se llama al método traerArrendadores y se verifica que el resultado no sea nulo y que contenga los arrendadores esperados.
- **Resultado Esperado:** El método traerArrendadores devuelve una lista de arrendadores no nula que contiene los arrendadores esperados.

- **testTraerArrendador**

- **Propósito:** Verificar que el método traerArrendador devuelva correctamente un arrendador existente.
- **Proceso:** Se crea un objeto Arrendador y se configura el repositorio para devolver este objeto cuando se busca por su ID. Se configura el modelMapper para mapear el objeto Arrendador a un objeto DTOArrendador. Luego, se llama al método traerArrendador con el ID del arrendador y se verifica que el resultado no sea nulo y que el repositorio haya buscado el arrendador por su ID.

- **Resultado Esperado:** El método traerArrendador devuelve un objeto DTOArrendador no nulo y el repositorio busca el arrendador por su ID.
- **testTraerArrendadorNotFound**
 - **Propósito:** Verificar que el método traerArrendador lance una excepción cuando el arrendador no se encuentra.
 - **Proceso:** Se configura el repositorio para devolver un Optional.empty() cuando se busca un arrendador por su ID. Luego, se llama al método traerArrendador con el ID del arrendador y se espera que lance una EntityNotFoundException. Se verifica que el repositorio haya buscado el arrendador por su ID.
 - **Resultado Esperado:** El método traerArrendador lanza una EntityNotFoundException y el repositorio busca el arrendador por su ID.
- **testCrearArrendador**
 - **Propósito:** Verificar que el método crearArrendador cree correctamente un nuevo arrendador.
 - **Proceso:** Se crea un objeto DTOArrendador y se configura el repositorio para devolver false cuando se verifica si el usuario ya existe. Se configura el modelMapper para mapear el objeto DTOArrendador a un objeto Arrendador. Luego, se configura el repositorio para guardar el objeto Arrendador y devolverlo. Se llama al método crearArrendador con el objeto DTOArrendador y se verifica que el resultado no sea nulo y que el repositorio haya verificado la existencia del usuario, guardado el arrendador y que el modelMapper haya realizado los mapeos correspondientes.
 - **Resultado Esperado:** El método crearArrendador devuelve un objeto DTOArrendador no nulo y el repositorio y modelMapper realizan las operaciones esperadas.
- **testEliminarArrendador**
 - **Propósito:** Verificar que el método eliminarArrendador elimine correctamente un arrendador existente.
 - **Proceso:** Se crea un objeto Arrendador y se configura el repositorio para devolver este objeto cuando se busca por su ID. Luego, se llama al método eliminarArrendador con el ID del arrendador y se verifica que el repositorio haya buscado el arrendador por su ID y que haya eliminado el arrendador encontrado.
 - **Resultado Esperado:** El repositorio busca el arrendador por su ID y elimina el arrendador encontrado.
- **testEliminarArrendadorNotFound**

- **Propósito:** Verificar que el método eliminarArrendador lance una excepción cuando el arrendador no se encuentra.
- **Proceso:** Se configura el repositorio para devolver un Optional.empty() cuando se busca un arrendador por su ID. Luego, se llama al método eliminarArrendador con el ID del arrendador y se espera que lance una EntityNotFoundException. Se verifica que el repositorio haya buscado el arrendador por su ID.
- **Resultado Esperado:** El método eliminarArrendador lanza una EntityNotFoundException y el repositorio busca el arrendador por su ID.

- **ServicioArrendatarioTest**

La clase ServicioArrendatarioTest contiene pruebas unitarias para los métodos de la clase ServicioArrendatario. Estas pruebas verifican la correcta funcionalidad de los métodos de la clase, asegurando que se comporten como se espera en diferentes escenarios.

- **testGetAllArrendatarios**

- **Propósito:** Verificar que el método getAllArrendatarios devuelva correctamente una lista de arrendatarios existentes.
- **Proceso:** Se crean varios objetos Arrendatario y se configura el repositorio para devolver una lista que contiene estos objetos cuando se llama al método findAll. Luego, se llama al método getAllArrendatarios y se verifica que el resultado no sea nulo y que contenga los arrendatarios esperados.
- **Resultado Esperado:** El método getAllArrendatarios devuelve una lista de arrendatarios no nula que contiene los arrendatarios esperados.

- **testGetArrendatarioById**

- **Propósito:** Verificar que el método getArrendatarioById devuelva correctamente un arrendatario existente.
- **Proceso:** Se crea un objeto Arrendatario y se configura el repositorio para devolver este objeto cuando se busca por su ID. Se configura el modelMapper para mapear el objeto Arrendatario a un objeto DTOArrendatario. Luego, se llama al método getArrendatarioById con el ID del arrendatario y se verifica que el resultado no sea nulo y que el repositorio haya buscado el arrendatario por su ID.
- **Resultado Esperado:** El método getArrendatarioById devuelve un objeto DTOArrendatario no nulo y el repositorio busca el arrendatario por su ID.

- **testGetArrendatarioByIdNotFound**

- **Propósito:** Verificar que el método getArrendatarioById lance una excepción cuando el arrendatario no se encuentra.

- **Proceso:** Se configura el repositorio para devolver un `Optional.empty()` cuando se busca un arrendatario por su ID. Luego, se llama al método `getArrendatarioById` con el ID del arrendatario y se espera que lance una `EntityNotFoundException`. Se verifica que el repositorio haya buscado el arrendatario por su ID.
- **Resultado Esperado:** El método `getArrendatarioById` lanza una `EntityNotFoundException` y el repositorio busca el arrendatario por su ID.
- **testCreateArrendatario**
 - **Propósito:** Verificar que el método `createArrendatario` cree correctamente un nuevo arrendatario.
 - **Proceso:** Se crea un objeto `DTOArrendatario` y se configura el repositorio para devolver `false` cuando se verifica si el usuario ya existe. Se configura el `modelMapper` para mapear el objeto `DTOArrendatario` a un objeto `Arrendatario`. Luego, se configura el repositorio para guardar el objeto `Arrendatario` y devolverlo. Se llama al método `createArrendatario` con el objeto `DTOArrendatario` y se verifica que el resultado no sea nulo y que el repositorio haya verificado la existencia del usuario, guardado el arrendatario y que el `modelMapper` haya realizado los mapeos correspondientes.
 - **Resultado Esperado:** El método `createArrendatario` devuelve un objeto `DTOArrendatario` no nulo y el repositorio y `modelMapper` realizan las operaciones esperadas.
- **testDeleteArrendatario**
 - **Propósito:** Verificar que el método `deleteArrendatario` elimine correctamente un arrendatario existente.
 - **Proceso:** Se crea un objeto `Arrendatario` y se configura el repositorio para devolver este objeto cuando se busca por su ID. Luego, se llama al método `deleteArrendatario` con el ID del arrendatario y se verifica que el repositorio haya buscado el arrendatario por su ID y que haya eliminado el arrendatario encontrado.
 - **Resultado Esperado:** El repositorio busca el arrendatario por su ID y elimina el arrendatario encontrado.
- **testDeleteArrendatarioNotFound**
 - **Propósito:** Verificar que el método `deleteArrendatario` lance una excepción cuando el arrendatario no se encuentra.
 - **Proceso:** Se configura el repositorio para devolver un `Optional.empty()` cuando se busca un arrendatario por su ID. Luego, se llama al método `deleteArrendatario` con el ID del arrendatario y se espera que lance una `EntityNotFoundException`. Se verifica que el repositorio haya buscado el arrendatario por su ID.

- **Resultado Esperado:** El método deleteArrendatario lanza una EntityNotFoundException y el repositorio busca el arrendatario por su ID.

- **ServicioCalificacionTest**

La clase ServicioCalificacionTest contiene pruebas unitarias para los métodos de la clase ServicioCalificacion. Estas pruebas verifican la correcta funcionalidad de los métodos de la clase, asegurando que se comporten como se espera en diferentes escenarios.

- **testTraerCalificaciones**

- **Propósito:** Verificar que el método traerCalificaciones devuelva correctamente una lista de calificaciones existentes.
- **Proceso:** Se crean varios objetos Calificacion y se configura el repositorio para devolver una lista que contiene estos objetos cuando se llama al método findAll. Luego, se llama al método traerCalificaciones y se verifica que el resultado no sea nulo y que contenga las calificaciones esperadas.
- **Resultado Esperado:** El método traerCalificaciones devuelve una lista de calificaciones no nula que contiene las calificaciones esperadas.

- **testTraerCalificacion**

- **Propósito:** Verificar que el método traerCalificacion devuelva correctamente una calificación existente.
- **Proceso:** Se crea un objeto Calificacion y se configura el repositorio para devolver este objeto cuando se busca por su ID. Se configura el modelMapper para mapear el objeto Calificacion a un objeto DTOCalificacion. Luego, se llama al método traerCalificacion con el ID de la calificación y se verifica que el resultado no sea nulo y que el repositorio haya buscado la calificación por su ID.
- **Resultado Esperado:** El método traerCalificacion devuelve un objeto DTOCalificacion no nulo y el repositorio busca la calificación por su ID.

- **testTraerCalificacionNotFound**

- **Propósito:** Verificar que el método traerCalificacion lance una excepción cuando la calificación no se encuentra.
- **Proceso:** Se configura el repositorio para devolver un Optional.empty() cuando se busca una calificación por su ID. Luego, se llama al método traerCalificacion con el ID de la calificación y se espera que lance una EntityNotFoundException. Se verifica que el repositorio haya buscado la calificación por su ID.
- **Resultado Esperado:** El método traerCalificacion lanza una EntityNotFoundException y el repositorio busca la calificación por su ID.

- **testCrearCalificacion**

- **Propósito:** Verificar que el método crearCalificacion cree correctamente una nueva calificación.
- **Proceso:** Se crea un objeto DTOCalificacion y se configura el modelMapper para mapear el objeto DTOCalificacion a un objeto Calificacion. Luego, se configura el repositorio para guardar el objeto Calificacion y devolverlo. Se llama al método crearCalificacion con el objeto DTOCalificacion y se verifica que el resultado no sea nulo y que el repositorio haya guardado la calificación y que el modelMapper haya realizado los mapeos correspondientes.
- **Resultado Esperado:** El método crearCalificacion devuelve un objeto DTOCalificacion no nulo y el repositorio y modelMapper realizan las operaciones esperadas.

- **testEliminarCalificacion**

- **Propósito:** Verificar que el método eliminarCalificacion elimine correctamente una calificación existente.
- **Proceso:** Se crea un objeto Calificacion y se configura el repositorio para devolver este objeto cuando se busca por su ID. Luego, se llama al método eliminarCalificacion con el ID de la calificación y se verifica que el repositorio haya buscado la calificación por su ID y que haya eliminado la calificación encontrada.
- **Resultado Esperado:** El repositorio busca la calificación por su ID y elimina la calificación encontrada.

- **testEliminarCalificacionNotFound**

- **Propósito:** Verificar que el método eliminarCalificacion lance una excepción cuando la calificación no se encuentra.
- **Proceso:** Se configura el repositorio para devolver un Optional.empty() cuando se busca una calificación por su ID. Luego, se llama al método eliminarCalificacion con el ID de la calificación y se espera que lance una EntityNotFoundException. Se verifica que el repositorio haya buscado la calificación por su ID.
- **Resultado Esperado:** El método eliminarCalificacion lanza una EntityNotFoundException y el repositorio busca la calificación por su ID.

- **ServicioPropiedadTest**

La clase ServicioPropiedadTest contiene pruebas unitarias para los métodos de la clase ServicioPropiedad. Estas pruebas verifican la correcta funcionalidad de los métodos de la clase, asegurando que se comporten como se espera en diferentes escenarios.

- **testGetAllPropiedades**

- **Propósito:** Verificar que el método `getAllPropiedades` devuelva correctamente una lista de propiedades existentes.
- **Proceso:** Se crean varios objetos `Propiedad` y se configura el repositorio para devolver una lista que contiene estos objetos cuando se llama al método `findAll`. Luego, se llama al método `getAllPropiedades` y se verifica que el resultado no sea nulo y que contenga las propiedades esperadas.
- **Resultado Esperado:** El método `getAllPropiedades` devuelve una lista de propiedades no nula que contiene las propiedades esperadas.
- **testGetPropiedadById**
 - **Propósito:** Verificar que el método `getPropiedadById` devuelva correctamente una propiedad existente.
 - **Proceso:** Se crea un objeto `Propiedad` y se configura el repositorio para devolver este objeto cuando se busca por su ID. Se configura el `modelMapper` para mapear el objeto `Propiedad` a un objeto `DTOPropiedad`. Luego, se llama al método `getPropiedadById` con el ID de la propiedad y se verifica que el resultado no sea nulo y que el repositorio haya buscado la propiedad por su ID.
 - **Resultado Esperado:** El método `getPropiedadById` devuelve un objeto `DTOPropiedad` no nulo y el repositorio busca la propiedad por su ID.
- **testGetPropiedadByIdNotFound**
 - **Propósito:** Verificar que el método `getPropiedadById` lance una excepción cuando la propiedad no se encuentra.
 - **Proceso:** Se configura el repositorio para devolver un `Optional.empty()` cuando se busca una propiedad por su ID. Luego, se llama al método `getPropiedadById` con el ID de la propiedad y se espera que lance una `EntityNotFoundException`. Se verifica que el repositorio haya buscado la propiedad por su ID.
 - **Resultado Esperado:** El método `getPropiedadById` lanza una `EntityNotFoundException` y el repositorio busca la propiedad por su ID.
- **testCreatePropiedad**
 - **Propósito:** Verificar que el método `createPropiedad` cree correctamente una nueva propiedad.
 - **Proceso:** Se crea un objeto `DTOPropiedad` y se configura el `modelMapper` para mapear el objeto `DTOPropiedad` a un objeto `Propiedad`. Luego, se configura el repositorio para guardar el objeto `Propiedad` y devolverlo. Se llama al método `createPropiedad` con el objeto `DTOPropiedad` y se verifica que el resultado no sea nulo y que el repositorio haya guardado la propiedad y que el `modelMapper` haya realizado los mapeos correspondientes.

- **Resultado Esperado:** El método createPropiedad devuelve un objeto DTOPropiedad no nulo y el repositorio y modelMapper realizan las operaciones esperadas.

- **testDeletePropiedad**

- **Propósito:** Verificar que el método deletePropiedad elimine correctamente una propiedad existente.
- **Proceso:** Se crea un objeto Propiedad y se configura el repositorio para devolver este objeto cuando se busca por su ID. Luego, se llama al método deletePropiedad con el ID de la propiedad y se verifica que el repositorio haya buscado la propiedad por su ID y que haya eliminado la propiedad encontrada.
- **Resultado Esperado:** El repositorio busca la propiedad por su ID y elimina la propiedad encontrada.

- **testDeletePropiedadNotFound**

- **Propósito:** Verificar que el método deletePropiedad lance una excepción cuando la propiedad no se encuentra.
- **Proceso:** Se configura el repositorio para devolver un Optional.empty() cuando se busca una propiedad por su ID. Luego, se llama al método deletePropiedad con el ID de la propiedad y se espera que lance una EntityNotFoundException. Se verifica que el repositorio haya buscado la propiedad por su ID.
- **Resultado Esperado:** El método deletePropiedad lanza una EntityNotFoundException y el repositorio busca la propiedad por su ID.

- **ServicioSolicitudTest**

La clase ServicioSolicitudTest contiene pruebas unitarias para los métodos de la clase ServicioSolicitud. Estas pruebas verifican la correcta funcionalidad de los métodos de la clase, asegurando que se comporten como se espera en diferentes escenarios.

- **testGetAllSolicitudes**

- **Propósito:** Verificar que el método getAllSolicitudes devuelva correctamente una lista de solicitudes existentes.
- **Proceso:** Se crea un objeto Solicitud y se configura el repositorio para devolver una lista que contiene este objeto cuando se llama al método findAll. Se configura el modelMapper para mapear el objeto Solicitud a un objeto DTOSolicitud. Luego, se llama al método getAllSolicitudes y se verifica que el resultado no sea nulo y que contenga las solicitudes esperadas.
- **Resultado Esperado:** El método getAllSolicitudes devuelve una lista de solicitudes no nula que contiene las solicitudes esperadas.

- **testGetSolicitudById**

- **Propósito:** Verificar que el método getSolicitudById devuelva correctamente una solicitud existente.
- **Proceso:** Se crea un objeto Solicitud y se configura el repositorio para devolver este objeto cuando se busca por su ID. Se configura el modelMapper para mapear el objeto Solicitud a un objeto DTOSolicitud. Luego, se llama al método getSolicitudById con el ID de la solicitud y se verifica que el resultado no sea nulo y que el repositorio haya buscado la solicitud por su ID.
- **Resultado Esperado:** El método getSolicitudById devuelve un objeto DTOSolicitud no nulo y el repositorio busca la solicitud por su ID.

- **testGetSolicitudByIdNotFound**

- **Propósito:** Verificar que el método getSolicitudById lance una excepción cuando la solicitud no se encuentra.
- **Proceso:** Se configura el repositorio para devolver un Optional.empty() cuando se busca una solicitud por su ID. Luego, se llama al método getSolicitudById con el ID de la solicitud y se espera que lance una EntityNotFoundException. Se verifica que el repositorio haya buscado la solicitud por su ID.
- **Resultado Esperado:** El método getSolicitudById lanza una EntityNotFoundException y el repositorio busca la solicitud por su ID.

- **testCreateSolicitud**

- **Propósito:** Verificar que el método createSolicitud cree correctamente una nueva solicitud.
- **Proceso:** Se crea un objeto DTOSolicitud y se configura el modelMapper para mapear el objeto DTOSolicitud a un objeto Solicitud. Luego, se configura el repositorio para guardar el objeto Solicitud y devolverlo. Se llama al método createSolicitud con el objeto DTOSolicitud y se verifica que el resultado no sea nulo y que el repositorio haya guardado la solicitud y que el modelMapper haya realizado los mapeos correspondientes.
- **Resultado Esperado:** El método createSolicitud devuelve un objeto DTOSolicitud no nulo y el repositorio y modelMapper realizan las operaciones esperadas.

- **testActualizarSolicitud**

- **Propósito:** Verificar que el método actualizarSolicitud actualice correctamente una solicitud existente.
- **Proceso:** Se crea un objeto Solicitud y un objeto DTOSolicitud, y se configura el repositorio para devolver el objeto Solicitud cuando se busca por su ID. Se configura el modelMapper para mapear el objeto DTOSolicitud a Solicitud y

viceversa. Luego, se configura el repositorio para guardar el objeto Solicitud y devolverlo. Se llama al método actualizarSolicitud con el ID de la solicitud y el objeto DTOSolicitud, y se verifica que el resultado no sea nulo y que el repositorio haya guardado la solicitud actualizada y que el modelMapper haya realizado los mapeos correspondientes.

- **Resultado Esperado:** El método actualizarSolicitud devuelve un objeto DTOSolicitud no nulo y el repositorio y modelMapper realizan las operaciones esperadas.

- **testEliminarSolicitud_Existente**

- **Propósito:** Verificar que el método eliminarSolicitud elimine correctamente una solicitud existente.
- **Proceso:** Se configura el repositorio para devolver un objeto Solicitud cuando se busca por su ID. Luego, se llama al método eliminarSolicitud con el ID de la solicitud y se verifica que el repositorio haya eliminado la solicitud encontrada.
- **Resultado Esperado:** El repositorio busca la solicitud por su ID y elimina la solicitud encontrada.

- **ServicioUsuarioTest**

La clase ServicioUsuarioTest contiene pruebas unitarias para los métodos de la clase ServicioUsuario. Estas pruebas verifican la correcta funcionalidad de los métodos de la clase, asegurando que se comporten como se espera en diferentes escenarios.

- **testCrearUsuario_UsuarioExistente**

- **Propósito:** Verificar que el método crearUsuario lance una excepción cuando el usuario ya existe.
- **Proceso:** Se configura el repositorio para devolver true cuando se verifica si el usuario ya existe. Luego, se llama al método crearUsuario con un objeto DTOUsuario y se espera que lance una IllegalArgumentException con el mensaje "El Usuario con este usuario ya existe.". Se verifica que el repositorio no guarda el usuario.
- **Resultado Esperado:** El método crearUsuario lanza una IllegalArgumentException con el mensaje "El Usuario con este usuario ya existe." y el repositorio no guarda el usuario.

- **testActualizarUsuario**

- **Propósito:** Verificar que el método actualizarUsuario actualice correctamente un usuario existente.
- **Proceso:** Se crea un objeto DTOUsuario y se configura el repositorio para devolver un objeto Usuario cuando se busca por su ID. Se configura el

modelMapper para mapear el objeto DTOUsuario a Usuario y viceversa. Luego, se llama al método actualizarUsuario con el ID del usuario y el objeto DTOUsuario, y se verifica que el resultado no sea nulo y que el repositorio haya guardado el usuario actualizado.

- **Resultado Esperado:** El método actualizarUsuario devuelve un objeto DTOUsuario no nulo y el repositorio guarda el usuario actualizado.
- **testEliminarUsuario**
 - **Propósito:** Verificar que el método eliminarUsuario elimine correctamente un usuario existente.
 - **Proceso:** Se configura el repositorio para devolver un objeto Usuario cuando se busca por su ID. Luego, se llama al método eliminarUsuario con el ID del usuario y se verifica que el repositorio haya eliminado el usuario encontrado.
 - **Resultado Esperado:** El repositorio busca el usuario por su ID y elimina el usuario encontrado.